Post-Training a Query-Aware Router for FiDeLiS

Project Subplan for Router Post-Training

1 Starting Point

We assume access to a working pretrained router that maps a question to a discrete search policy for FiDeLiS. This router is already useful: it often predicts a *correct* policy in the sense that FiDeLiS can still answer the question, but it does not consistently predict the *best* policy for the downstream search. Here, "best" means the policy that gives the best trade-off between answer quality, path faithfulness, and computation cost.

The post-training stage therefore does *not* create a supervised oracle dataset of best hyperparameters. Instead, it treats FiDeLiS as an online environment. For each query, the router proposes or samples a search policy, FiDeLiS executes that policy, and the resulting answer/path/cost measurements are turned into a scalar reward.

2 Problem Setup

Let x be the natural-language query and let $a \in \mathcal{A}$ be a discrete search policy. Each action a corresponds to a bundle of FiDeLiS search settings such as maximum depth, beam width, candidate width, future-lookahead strength, and local path constraints. The pretrained router provides prior logits $\pi_0(a \mid x)$ over the action set. Post-training learns an improved router that adapts these prior preferences using online reward from FiDeLiS.

Because the router makes one routing decision per query before search begins, the task is naturally modeled as a **contextual bandit** rather than a full multi-step MDP. This matters methodologically: we do not need to learn long-horizon temporal credit assignment inside the router, because FiDeLiS itself executes the search after the action is chosen.

3 Reward

For a sampled action a on query x, FiDeLiS returns a final prediction, a set of reasoning paths, and search statistics. We compute a scalar reward as a weighted mean of bounded metrics rather than as a sum of logits or logistic outputs. A representative reward is

$$R(x, a) = w_{\text{hit}} \cdot \text{Hit} + w_{\text{f1}} \cdot \text{F1} + w_{\text{path}} \cdot \text{PathSupport} + w_{\text{eff}} \cdot \text{Efficiency},$$

where:

- $\text{Hit} \in \{0, 1\}$ indicates whether the final answer is correct.
- $\text{F1} \in [0, 1]$ gives partial credit on multi-answer questions.
- $\text{PathSupport} \in [0, 1]$ measures agreement between predicted reasoning paths and available gold paths or gold supporting facts.

- Efficiency $\in [0, 1]$ penalizes expensive policies through runtime, number of expansions, and number of LLM calls.

This reward is directly computable from the FiDeLiS pipeline and does not require differentiability. The router only needs a scalar training signal.

4 Method 1: Value-Based Router

4.1 Idea

The value-based router learns a function $Q_\theta(x, a)$ that predicts the expected FiDeLiS reward if action a is chosen for query x. At inference time, it selects

$$a^\star = \arg\max_{a \in \mathcal{A}} Q_\theta(x, a).$$

4.2 Why this method

This is the strongest value-style baseline for our setting because the problem is a one-step decision over a small discrete action set. We do not need Bellman bootstrapping over long trajectories; we only need to regress observed returns for chosen actions.

4.3 Small theory section

In a contextual bandit, the optimal policy is determined by the action with the highest conditional expected reward:

$$\pi^\star(x) = \arg\max_a \mathbb{E}[R \mid x, a].$$

If $Q_\theta(x, a)$ is a good estimator of this quantity, greedy action selection is Bayes-optimal with respect to the learned model. In practice, we learn Q_θ by online regression on observed tuples (x, a, R) collected during exploration.

4.4 How we use the pretrained router

The pretrained router supplies prior logits over actions. We do not discard them. Instead, we use them as prior features and let the value head learn a better estimate of downstream reward. This is appropriate when the pretrained model is usually reasonable but not yet optimal.

5 Method 2: Ranking-Based Router

5.1 Idea

The ranking-based router learns a utility score $U_\theta(x, a)$ and trains on *relative* preferences between actions for the same query. If action a_i gives higher observed reward than a_j on the same query, the model is updated to rank a_i above a_j.

5.2 Why this method

Ranking is appealing when absolute rewards are noisy, scale-sensitive, or variable across datasets. The main question for routing is often not "what is the precise reward?" but rather "which of these policies is better for this query?" Pairwise preference learning targets that question directly.

5.3 Small theory section

We model pairwise preference probabilities with a Bradley–Terry style form:

$$P(a_i \succ a_j \mid x) = \sigma(U_\theta(x, a_i) - U_\theta(x, a_j)),$$

where σ is the logistic sigmoid. Given two actions tried on the same query, we define the preferred action using the observed FiDeLiS reward and optimize a pairwise logistic loss. This turns noisy scalar feedback into a more robust ordering objective.

5.4 How we use the pretrained router

Again, the pretrained router acts as an informative prior. The ranking head can be initialized from, or conditioned on, the pretrained action logits. The learned utility then re-orders policies using online FiDeLiS outcomes.

6 Method 3: Bandit-Style RL Fine-Tuning

6.1 Idea

The bandit RL router directly fine-tunes the action distribution $\pi_\theta(a \mid x)$ using policy gradient. We sample an action, execute FiDeLiS, obtain reward R, and update the router with a one-step policy objective.

6.2 Why this method

This is the most direct way to optimize the actual routing objective without constructing oracle labels. It is especially useful once the pretrained router is already good, because RL can make small policy adjustments toward higher reward while respecting the prior.

6.3 Small theory section

Because each episode consists of a single decision, REINFORCE reduces to

$$\nabla_\theta J(\theta) = \mathbb{E}_{a \sim \pi_\theta(\cdot \mid x)} \left[(R - b) \nabla_\theta \log \pi_\theta(a \mid x) \right],$$

where b is a baseline used to reduce variance. We additionally regularize the updated policy toward the pretrained router so that fine-tuning stays close to a model that is already mostly correct. This is more suitable here than treating the problem as a full PPO-style trajectory optimization problem, since the routing decision is fundamentally one-step.

6.4 Relation to the other two methods

The value-based router predicts expected reward. The ranking-based router predicts relative utility. The bandit RL router optimizes the policy distribution itself. Together they form a clean comparison across three distinct post-training styles.

7 Experimental Plan

7.1 Action space

Each action corresponds to a policy bundle over FiDeLiS arguments:

- search depth (‘max_length‘);
- beam width (‘top_k‘);
- candidate width (‘top_n‘);
- future-score weight (‘alpha‘);
- lookahead depth (‘lookahead_hops‘);
- local search constraints such as no immediate backtracking and simple-path enforcement.

The action set stays small and discrete so that online exploration remains computationally feasible.

7.2 Training protocol

We use only the raw train split of the project dataset and online FiDeLiS reward. No supervised oracle routing dataset is created. For each method:

1. initialize from the pretrained router outputs;
2. explore actions online on training queries;
3. compute reward from FiDeLiS outputs;
4. update the corresponding post-training head;
5. evaluate greedily on held-out validation and test splits.

7.3 Metrics

We report:

- answer Hit and F1;
- path-support / faithfulness proxy;
- average runtime;
- average number of LLM calls;
- average number of path expansions;
- scalar reward.

The scalar reward is the training objective; the metric table shows the underlying trade-offs.

7.4 Ablations

The router post-training component will be compared under:

- no router (original FiDeLiS fixed policy);
- pretrained router without post-training;
- value-based post-training;
- ranking-based post-training;
- bandit RL fine-tuning;
- without query-constraint inputs;
- without no-backtracking / simple-path search constraints.

8 Expected Outcome

The main hypothesis is not that one universal policy dominates all queries. Rather, different queries benefit from different search budgets and constraints, and a pretrained router can be improved by post-training directly against FiDeLiS reward. We expect:

- the value-based router to be stable and sample-efficient;
- the ranking-based router to be robust when reward magnitudes are noisy;
- bandit-style RL to produce the best final reward once initialized from a competent pretrained model.